

❏ CHAPTER 2: VM PLATFORM, DATA CONVERSIONS & BOOLEAN LOGIC

▼ Why Low-Level Knowledge Matters (Especially for C/C++ Devs)

If you're serious about becoming a **C or C++ developer**, you *can't ignore* what's happening under the hood. These languages give you power and control — but also more chances to shoot yourself in the foot if you don't understand memory, instructions, or the system architecture.

🧠 High-level languages hide a lot from you.

💣 C/C++ forces you to face reality — memory, pointers, addresses, machine code.

When bugs happen in C or C++, they often *don't show up clearly* at the source code level. You may need to “drill down” into the:

- **CPU registers**
- **Raw memory**
- **Instruction-level behavior**

That's where tools like **assemblers**, **linkers**, and **debuggers** come in.

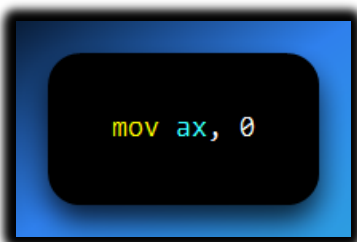
🔧 Core Tools of the Low-Level World

1) Assembler — Translating Assembly to Machine Code

An **assembler** takes your **assembly language** code and converts it into **machine code** — the binary stuff the CPU can actually execute.

- Input: **.asm** or **.s** file (your code)
- Output: **.obj** or **.o** file (machine-readable object code)

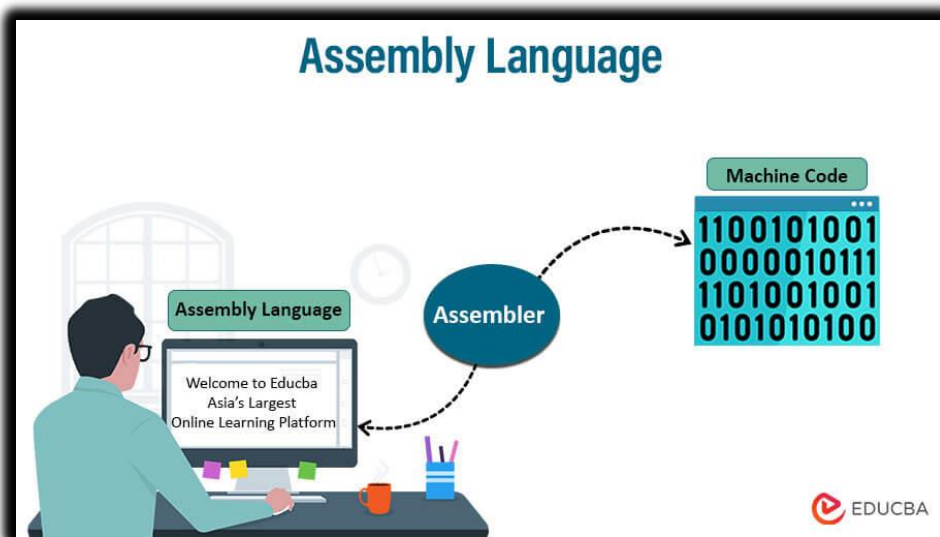
Example:



Assembler turns this into something like:



Assemblers are like compilers, but for assembly. They don't optimize — they just **translate**.



2) Linker — Merging the Pieces

A **linker** takes multiple object files and glues them together into a final **executable**.

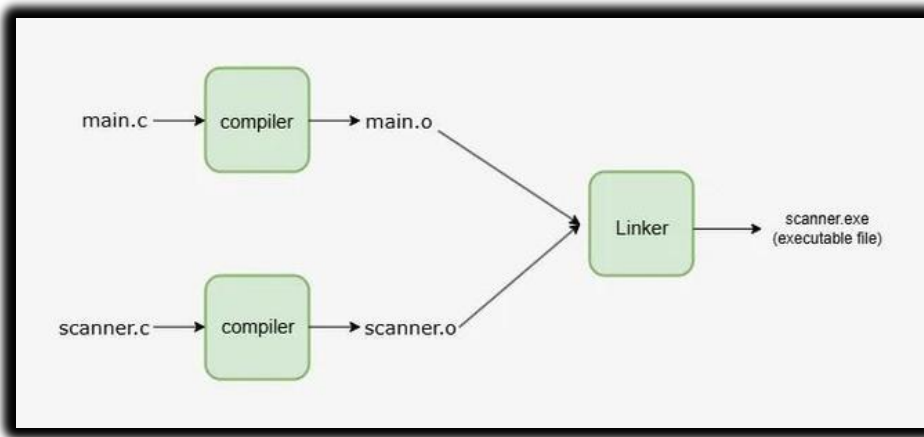
Why do we need it?

- Programs are often split into multiple files/modules
- Some parts (like libraries) come from external sources
- The linker resolves things like:
 - **Function calls** across files
 - **Global variable references**
 - **External libraries**

Input: **.obj** files

Output: **.exe** or **.elf** or **.bin** (platform-specific)

💡 Think of the linker as your program's **final assembler**, stitching all parts into one runnable body.



3) Debugger — Your X-Ray Vision Tool

A debugger lets you:

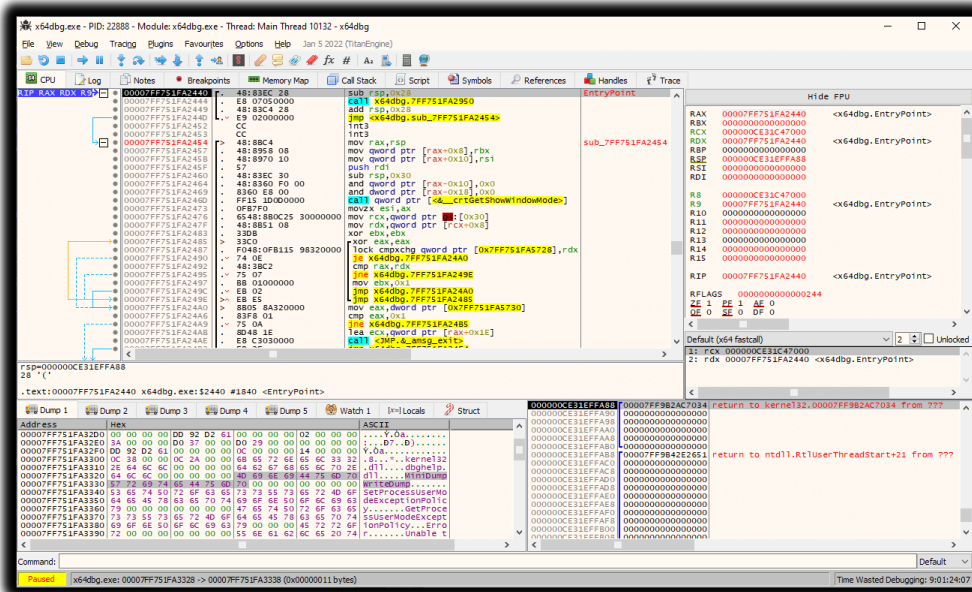
- **Pause** your program at specific points (breakpoints)
- **Step through** code line-by-line
- **Inspect registers, memory, variables**, and the **call stack**
- Find the exact moment things go wrong

This is how pros figure out:

- Where segmentation faults happen
- Why a variable has a garbage value
- Whether a function even got called

Popular debuggers:

- gdb (GNU Debugger — Linux CLI)
- Visual Studio Debugger (GUI-based)
- WinDbg (Windows kernel-level debugging)



TOOL

PURPOSE

INPUT → OUTPUT

Assembler

Converts your human-readable assembly code (like MOV, ADD) into raw machine code (binary 1s and 0s) that the CPU understands.

.asm (assembly source file) → .obj (object file)

Linker

Combines one or more object files (from the assembler) and any necessary external libraries into a single, final executable program or library. It resolves references between different code modules.

.obj (object files) +
.lib (libraries) →
.exe (executable) or
.bin (binary)

Debugger

A powerful tool that lets you pause your program, step through its execution line by line, inspect memory, examine register values, and identify bugs. It's like a microscope for your code.

Executable +
Symbols
(debugging info)

Yes — *loaders* are the last critical piece of the “code-to-execution pipeline” that most people skip.

If **assemblers** translate, **linkers** stitch, then **loaders** are the ones that *launch the beast into RAM*. Let’s plug that in and wrap this whole section up neatly before moving forward.

Loaders — The Final Step: Getting Code into Memory

What is a Loader?

A **loader** is a system-level program that takes your fully-linked executable (like .exe or .out) and loads it into **RAM** for execution.

Think of it like the valet:

You hand over the car keys (the executable), and the loader parks the program into memory, sets up the environment, and gives control to the CPU to start execution.

What the Loader Actually Does:

1. **Reads the executable file:** Loads headers, segments, and metadata.
2. **Allocates memory** in RAM for code, data, and stack.
3. **Loads code & data sections** into correct memory locations.
4. **Resolves runtime dynamic links** (e.g. to shared DLLs or .so files).
5. Sets up the **stack, heap, and registers**
6. **Starts execution** by jumping to the program's *entry point* (usually `main()`)

Without the loader, your code would just sit there in storage — not running, not alive.

Difference between linkers and loaders:

FEATURE	LINKER	LOADER
When it runs	Before execution (at compile/build time)	At runtime (when you launch the program)
What it does	Combines object files (.obj) and static libraries into a single, final executable binary (like .exe or .bin). It's building the complete package.	Loads that final executable binary from disk into the computer's RAM (memory) and prepares it for execution by the CPU. It's putting the package into action.
Resolves	External symbols: This means it finds the actual memory addresses for functions and global variables that are defined in other parts of your code or in static libraries. It's like filling in all the "Go to page X" references in a book.	Dynamic libraries (also known as shared libraries or DLLs on Windows): It finds and loads shared runtime code that the program needs, but which isn't part of the main executable. It's like bringing in guest speakers (libraries) only when their expertise is needed.
Output	A complete, ready-to-run executable file on disk.	A running process in memory, executing your program's instructions.

```
Source Code → Assembler → Object Code
Object Code → Linker → Executable
Executable → Loader → Running Program (in RAM)
```

Why You Should Care as a Developer

- Knowing what the loader does helps you debug **runtime crashes**, **missing DLLs**, or **initialization errors**
- Low-level tools (like manual shellcode injection, OS dev, reverse engineering) rely heavily on **manual loading**
- Understanding the loader's job clarifies how your .exe or .out goes from **disk** → **memory** → **CPU**

Alright, now that we've locked in the **full pipeline from writing code to executing it**, you've got the full picture:

Source code → Assembler → Linker → Loader → CPU runs it.

These tools give you deep control over the program, allowing for:

- Efficient code generation
- Tight memory usage
- Advanced debugging and reverse engineering

FILE TYPES AND CPU MODES SUPPORTED BY MASM (MICROSOFT MACRO ASSEMBLER)

MASM creates **different types of output files** depending on the **CPU mode** your code is written for. These modes correspond to processor architectures (16-bit, 32-bit, and 64-bit) and affect how your assembly code is written and executed.

◆ **16-Bit Real-Address Mode**

- Legacy mode used in DOS and embedded systems.
- Not supported on modern 64-bit Windows OS without emulators.
- Will only be covered in Chapters 14–17 for historical context.

◆ **32-Bit Protected Mode**

- Supported by all 32-bit Windows OS versions.
- Easier to write and debug than 16-bit real mode.
- This is what most MASM tutorials use as the default.

 We'll refer to this as just “32-bit mode” from now on.

◆ **64-Bit Long Mode**

- Runs on all 64-bit versions of Windows.
- Requires use of different registers (RAX, RBX, etc.) and calling conventions.
- MASM64 or x64 assembly tools are needed.

Let's go deep on the above, I can't leave my beginners hanging:

✅ Target: You'll finally understand what MASM is, what it creates, what "CPU modes" even mean, and why it all matters when writing assembly code.

✅ No more guessing. No more reading the same paragraph 5 times like "wait huh?" 🤖

🔑 What is MASM?

MASM stands for **Microsoft Macro Assembler**.

It's a program that **takes your assembly code** (with commands like **mov eax, 1**) and **converts it into machine code** — the raw binary instructions the CPU actually runs.

In other words:

- You write readable low-level code → MASM turns it into .obj (object) files or .exe (executable) files.
- This is just like how gcc compiles C code into machine code — but MASM does it for Assembly.

📁 What File Types Does MASM Create?

FILE TYPE	WHAT IT IS	WHEN IT APPEARS
.ASM	Your original source file. This is where you write your assembly code using human-readable mnemonics and directives. It's just a plain text file.	You write this file yourself, typically using a text editor or IDE.
.OBJ	An object file. This is the output of the assembler (like MASM). It contains the raw machine code for your program, but it's not yet a complete, runnable program. It might have unresolved references to functions or data in other files.	Created by MASM (or another assembler) after it successfully processes your .asm source code.
.EXE	An executable program. This is the final, runnable file that your operating system can load and execute. It's created by a linker, which combines one or more .obj files and any necessary libraries into a single, complete package.	Created by the linker (like LINK.EXE) after it successfully combines your .obj files and resolves all external references.

💡 **Key takeaway:** MASM (the assembler) only does the .ASM → .OBJ part. You need a separate tool called a **linker** (like LINK.EXE on Windows) to go from .OBJ → .EXE.

🔧 MASM vs Other Assemblers — They're All Translators

Let's compare MASM to other assemblers:

Assembler	Who Made It	Popular On	Notable For
MASM	Microsoft	Windows	Powerful macros, Windows development
NASM	Open source	Linux, Cross-platform	Clean syntax, very beginner-friendly
FASM	Independent devs	Windows, Linux	Compact executables, speed
GAS	GNU (open source)	Linux, Unix systems	Used with GCC (.s files), older syntax

Full description in the html and the image.

They all **convert assembly into machine code**.

The only difference is syntax, platform, and ecosystem preferences.

💬 **Think of it like this:**

- MASM is like **American English**
- NASM is like **British English**
- GAS is like **Shakespearean English**
- FASM is like **minimalist text-speak**

They all **say the same thing**, just in slightly different dialects.

Why Should You Care Which One?

If you're targeting:

- **Windows dev / WinAPI / legacy x86 tutorials** → MASM is your friend
- **Cross-platform / Linux / modern low-level** → NASM or GAS is better
- **Extreme performance & size optimization** → FASM might appeal to you

But the fundamentals of Assembly language don't change — registers are registers, instructions are instructions, and you're still doing the same CPU-level work.

You now fully understand:

- What MASM is
- What it produces
- How it compares to NASM/FASM/GAS
- Why it matters when choosing your tool

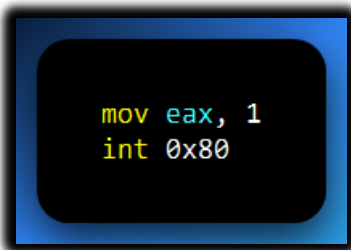
File Types Created by Assemblers: From Code to Executable

Assembly code doesn't run by itself. You write source code, then the assembler, linker, and maybe even other tools step in to create something the CPU can actually execute.

Let's break it down — beginner-friendly, real-world style:

.ASM — Your Source File

This is **your code**, written by you, in a human-readable format:



- It's just **plain text**, like a .txt file.
- You write it using MASM, NASM, VSCode, etc.
- It's not executable — it needs to be *assembled*.

.OBJ — Object File (Assembled, But Not Ready Yet)

Think of .OBJ as a **half-built car**. The engine's in, but the wheels and doors aren't connected yet.

- Created **by the assembler** from your .ASM file.
- Contains **machine code** — but some parts are still unresolved (like calls to external functions).
- Still **not runnable** — you need to link it.

Real analogy:

It's like you've prepped all your ingredients and chopped your veggies, but you haven't cooked the dish yet.

.EXE — Executable File (Ready to Run)

This is the final boss — the file you can actually **double-click** in Windows and watch it run.

- Created **by a linker**, not the assembler.
- Combines one or more .OBJ files + any required libraries (like Windows API).
- Links up external references (like printf, CreateFile, etc.).
- Adds sections like .text (code), .data (vars), .rdata (constants), etc.

Real analogy:

The linker takes all the car parts from .OBJ and builds a full car you can drive.

.BIN — Raw Binary File (No Frills, No OS)

This is **bare metal** — a file that's meant to be run **without an OS**.

- Just raw bytes — pure machine code, no headers or OS-specific structures.
- Often used in:
 - Bootloaders.
 - BIOS firmware.
 - Embedded devices (like Arduino, STM32).
- Doesn't work like .EXE — it gets **flashed** to chips or run by a boot ROM.

Real analogy:

Imagine taking your machine code and tattooing it straight onto a CPU — no fancy packaging, no instructions.

A .BIN file is just raw data, with no extra formatting or system information.

Why This Matters for Reverse Engineering

When you're reverse engineering, you're usually handed a mysterious .EXE, .DLL, or .BIN. Understanding how that file was **built** helps you know what you're dealing with:

File Type	What You'll See Inside
.EXE	Sections, imports, function names, resources
.BIN	Raw machine code, maybe a boot sector, no structure
.OBJ	Unlinked machine code, sometimes helpful for RE libs

Knowing whether a file was created by **MASM**, **GCC**, or **NASM** also gives clues about calling conventions, section layouts, and symbol naming.

File Type	What It Is	Who Creates It	Usable For...
.ASM	Your source code	You	Writing assembly code
.OBJ	Assembled machine code (not linked)	Assembler (e.g. MASM)	Linking later
.EXE	Final executable	Linker	Running in Windows
.BIN	Raw machine code	Assembler (raw mode)	Flashing to devices, bootloaders

After you read this, I wanted to keep going on CPU modes, but damn! 🤪 That topic is hot, I can't just toss it in here, let's move to the [next document](#).

We shall cover all the CPU-Modes addressed in the [007.MASM.FileTypes.Operation.Modes.confusion.removed.html](#) document.

⚖️ Assembly Language vs. Machine Language

Let's compare these two side-by-side:

Find the [htmls](#) or [images](#) file attached for a complete table of comparison.

🧠 Summary:

- **Assembly** = **human-friendly** representation of machine code
- **Machine code** = **binary** the CPU executes directly
- Assembly is easier to *write*, *debug*, and *understand*
- Machine language is harder to work with, but runs at **maximum efficiency**

💬 Final Thoughts:

- Writing machine code by hand is like writing in Morse code with no mistakes allowed.
- Writing assembly is like shorthand — better than Morse, but still not English.
- Use assembly when you need **fine-tuned control**, **speed**, or **hardware access**.
- Use a debugger and linker to manage and inspect the full system-level workflow.

Assembly gives you **power and control**, but also **responsibility and danger**.

High-level languages give you **speed of development**, **maintainability**, and **portability**.

In today's world:

- Use **high-level languages** for most applications.
- Use **assembly** when you absolutely need **performance** or **hardware control**.

And now you understand why game engines, OS kernels, and malware authors still use this 40+ year-old beast. 💣

HarmonyOS

awesome

A curated list of awesome things related to HarmonyOS

I . What is HarmonyOS?

HarmonyOS is the first full-scene distributed OS based on microkernel. It is an operating system independently developed by Huawei. HarmonyOS System was officially released at the Huawei Developers Conference (<HDC.2019>) on August 9, 2019. Huawei will take the lead in deploying intelligent terminals such as smart screens, vehicle terminals and wearable terminals. In the future, more smart devices will be equipped with open source HarmonyOS than ever.

Packages

No packages published

Contributors 13

Languages

Language	Percentage
C	94.9%
Assembly	1.4%
Makefile	0.7%
Other	0.3%
Shell	1.5%
Perl	0.8%
C++	0.4%

Joe Grand

@JoeGrand · 451k subscribers · 253 videos

Hacking, electronics, and other projects featuring Joe Grand, a computer engineer, hardware hacker, and more. ...more

[joegrund.com](#) and 2 more links

Subscribe

Community

Home

Videos

Live

Playlists

Posts

Q

Latest

Popular

Oldest

The Shocking Result of My \$75 Million Bulk Hack

140k views · 2 days ago

\$75,000,000 Crypto Wallet Bulk Hack

388k views · 5 days ago

I hacked time to recover \$3 million from a Bitcoin software wallet

1.3m views · 1 year ago

THE WORLD'S THINNEST BOOMBOX

141k views · 3 years ago



samy kamkar ✓

@samykamkar · 203k subscribers · 15 videos

Hi! I'm Samy, and welcome to the Applied Hacking channel. ...more

[sa.my](#) and 1 more link

Subscribe

Home Videos Shorts Playlists Q

Latest Popular Oldest



Lollapalowski 2018
15k views · 7 years ago



PoisonTap - exploiting locked machines w/Raspberry Pi Zero
869k views · 9 years ago



MagSpooft - magnetic stripe spoofer / credit card magstripe emulator
461k views · 10 years ago



OwnStar - hacking cars with OnStar to locate, unlock and remote start...
265k views · 10 years ago



northwestrepair

@northwestrepair · 219k subscribers · 370 videos

Contact me on DISCORD ...more

[discord.gg/yXeM6eUjNe](#) and 1 more link

Subscribe Community

Home Videos Shorts Playlists Posts Q

Latest Popular Oldest



IR6500 Upgrades NOT FOR SALE
6.3k views · 20 hours ago



Zotac 3080Ti
19k views · 1 day ago



Is ZOTAC 5090 the best 5090 money can buy ?
15k views · 3 days ago



Gigabyte 5090 just randomly quit
29k views · 6 days ago

FEATURE	ASSEMBLY LANGUAGE	MACHINE LANGUAGE
Readability	Human-readable mnemonics (like MOV, ADD, JMP) and symbols. It's like shorthand notes for humans.	Pure binary (e.g., 10110000 00000001). Completely unreadable and meaningless to humans without a lookup table.
Complexity	Uses symbols, labels, and opcodes, making it more manageable for humans to write and understand than raw binary.	Just raw binary numbers. No labels, no symbols, just sequences of 1s and 0s that directly instruct the CPU.
Portability	Not directly portable across different CPU architectures (e.g., x86 assembly won't run on ARM). Each architecture has its own assembly language.	Also not portable. It's the native language of a specific CPU architecture and cannot be executed on a different type of CPU.
Maintenance	Easier to debug and modify compared to machine code, as you're working with symbolic instructions.	Extremely difficult to read, debug, or modify directly. Making changes requires converting binary back to assembly, modifying, and re-converting.
Efficiency	Very close to the hardware; allows for highly optimized and efficient code, as you have direct control over CPU operations.	The fastest possible execution, as it's the native format directly understood and executed by the CPU without any translation overhead.
Error-prone	High chance of human error due to the manual, low-level nature of writing every instruction. It's like writing every step of a dance routine by hand.	Machine-generated, so fewer human errors in its creation, but no flexibility. Errors in the source assembly lead to errors in the machine code.
Hardware-dependence	Tied directly to the specific CPU architecture's instruction set.	Same as assembly; it must match the hardware's instruction set exactly to be executable.
Size	Slightly larger in source form due to the use of mnemonics and comments.	More compact in its final form, as it's just raw binary data without any symbolic overhead.